

Capitolo 46 — PROT-017 — Persistent Mutation Safety

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-46
Data master	2026-09-02
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/46_prot_017_persistent_mutation_safety.md
Source SHA	1ff76e4
Release	2026-09-04T09:44:14.164Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

PARTE VII — Il Libro dei Protocolli WCM Stato: FROZEN **Data:** 2026-09-02 **Scope:** WCM generale, domain-agnostic

46.0 Scrivere è facile. Scrivere senza danneggiare ciò che esiste è un altro problema

Nei capitoli precedenti abbiamo visto che il WCM cerca di rendere affidabili alcune operazioni che non dovrebbero dipendere ogni volta da una nuova interpretazione. Ma c'è un momento ancora più delicato della lettura di uno stato: il momento in cui il sistema **modifica qualcosa che resterà modificato**.

Salvare un file, aggiornare uno stato, registrare un comando o sostituire una configurazione sembrano azioni semplici. Tecnicamente, spesso lo sono. Il problema è che una scrittura può riuscire perfettamente dal punto di vista tecnico e produrre comunque un risultato sbagliato.

Un sistema può infatti confermare: “scrittura completata”, mentre in realtà ha:

- modificato l'oggetto sbagliato;
- usato una versione ormai superata;
- cancellato informazioni che dovevano essere conservate;
- duplicato un effetto già prodotto;
- scritto contemporaneamente a un altro componente;
- applicato un comando oltre i limiti dell'autorità disponibile.

PROT-017 — Persistent Mutation Safety nasce per proteggere proprio questo passaggio.

Il suo principio fondamentale può essere espresso senza terminologia tecnica:

Prima di cambiare qualcosa che resterà cambiato, non basta sapere che possiamo scrivere. Dobbiamo essere certi di cosa stiamo cambiando, con quale diritto, rispetto a quale versione e con quale risultato effettivo.

46.1 Che cosa significa “mutazione persistente”

Una mutazione persistente è una modifica che sopravvive all'azione che l'ha prodotta.

Se prendiamo un foglio e annotiamo mentalmente una correzione senza salvarla, quella correzione non è ancora persistente. Se invece sostituiamo il documento condiviso, aggiorniamo uno stato operativo o registriamo un comando destinato a produrre effetti successivi, abbiamo modificato la memoria o il comportamento futuro del sistema.

Nel WCM questo concetto comprende, tra le altre cose:

- modifiche alla baseline o al canone;
- aggiornamenti dello stato operativo persistente;
- registrazione o consumo di authority;
- sostituzione di contenuti esistenti;
- aggiornamento di configurazioni condivise;
- scritture effettuate da automazioni su risorse comuni.

La parola importante non è quindi “file”. È **persistente**.

Il protocollo riguarda il rischio che una modifica durevole produca conseguenze che si propagano oltre l'istante in cui è stata eseguita.

46.2 Il problema più insidioso: una scrittura riuscita può essere sbagliata

Immaginiamo un archivio condiviso contenente la versione corrente di un documento. Un operatore legge la versione 7 e prepara una modifica. Nel frattempo un secondo operatore aggiorna correttamente il documento alla versione 8.

Il primo operatore, che non sa dell'aggiornamento, salva la propria modifica costruita sulla versione 7.

La scrittura può essere tecnicamente perfetta. Il sistema può restituire un messaggio di successo. Ma il risultato è una regressione: parte del lavoro della versione 8 è stata cancellata.

Questo è il motivo per cui PROT-017 distingue due domande:

LA SCRITTURA È RIUSCITA?

≠

LA SCRITTURA ERA CORRETTA E SICURA?

La seconda domanda richiede controlli prima e dopo l'operazione.

46.3 Recovery non è prevention

Uno dei principi più importanti del protocollo è:

| Poter recuperare un errore non equivale a prevenirlo.

Avere una cronologia delle versioni è prezioso. Disporre di backup, rollback e possibilità di retry riduce il danno quando qualcosa va storto. Ma nessuno di questi strumenti rende accettabile una scrittura non controllata.

È la differenza tra avere un estintore e costruire un impianto che riduca il rischio di incendio.

Servono entrambi, ma non svolgono la stessa funzione.

PROT-017 lavora soprattutto sulla prevenzione: cerca di impedire che una modifica ambigua, stale, concorrente, distruttiva o non autorizzata venga accettata come effetto valido.

46.4 Quando il protocollo entra in gioco

PROT-017 si applica prima di una scrittura persistente quando il rischio non è puramente locale o temporaneo.

Il protocollo diventa particolarmente importante quando la modifica riguarda una baseline condivisa, uno stato operativo, una authority, un contenuto già esistente o una risorsa sulla quale potrebbero agire più scrittori.

Un altro segnale importante è la propagazione: se un errore nella scrittura potrebbe essere letto successivamente da altri componenti, automazioni o viste, il rischio non termina nel punto in cui il dato viene salvato.

In termini semplici:

```
MODIFICA SOLO TEMPORANEA E LOCALE?  
→ protezioni proporzionate  
  
MODIFICA PERSISTENTE E CONDIVISA?  
→ PROT-017
```

Il protocollo non impone quindi lo stesso peso a ogni operazione. La protezione deve essere proporzionata al rischio.

46.5 Primo controllo: sto modificando esattamente la cosa giusta?

Prima di scrivere, il target deve essere identificato senza approssimazioni.

Se esistono più documenti con nomi simili, più versioni dello stesso oggetto o più ambiti possibili, il sistema non deve scegliere quello “che sembra giusto”.

Deve conoscere esattamente:

- la risorsa;
- l'ambito;
- l'identità logica;
- il confine entro cui la modifica è autorizzata.

Questo è l'**Exact Target / Scope Validation**.

La regola è semplice:

```
TARGET AMBIGUO  
→ NO WRITE
```

Non significa che l'attività sia impossibile. Significa che prima di modificare qualcosa occorre eliminare l'ambiguità.

46.6 Secondo controllo: ciò che sto per scrivere ha una forma valida?

Sapere dove scrivere non basta. Bisogna anche controllare **che cosa** stiamo scrivendo.

Una modifica persistente può avere campi obbligatori, valori ammessi, relazioni interne o altre condizioni che devono essere rispettate. Se manca un elemento essenziale, accettare comunque il contenuto può creare uno stato che esiste fisicamente ma non è utilizzabile in modo sicuro.

PROT-017 richiede quindi un controllo del payload e dello schema quando questi esistono.

In termini non tecnici: prima di consegnare un modulo importante non controlliamo soltanto l'indirizzo del destinatario; controlliamo anche che il modulo sia completo e coerente.

Valore sconosciuto o struttura incompleta non autorizzano una correzione creativa. Quando il significato operativo non è determinabile in modo univoco, il comportamento corretto è fermarsi.

46.7 Terzo controllo: sto ancora lavorando sulla versione corrente?

Questo è uno dei punti centrali del protocollo.

Una modifica può essere corretta rispetto alla fotografia che abbiamo letto e sbagliata rispetto alla realtà che esiste nel momento della scrittura.

Per evitarlo, PROT-017 usa un **Expected Version / State Guard**.

Il principio è:

“Scrivi questa modifica solo se la risorsa è ancora quella sulla quale ho costruito il mio lavoro.”

La verifica può essere rappresentata attraverso una versione, un'impronta, una revisione o un altro indicatore affidabile dello stato atteso.

Se la realtà è cambiata nel frattempo:

```
EXPECTED STATE ≠ CURRENT STATE  
→ NON FORZARE  
→ RILEGGI  
→ RICALCOLA / REPLAY
```

Questo impedisce che una modifica vecchia sovrascriva silenziosamente una modifica più recente.

46.8 Quarto controllo: chi ha il diritto di scrivere qui?

Quando più componenti possono modificare la stessa risorsa contemporaneamente, il risultato può dipendere dall'ordine casuale delle scritture.

PROT-017 riprende quindi un principio già incontrato parlando di stato deterministico: quando possibile, una risorsa condivisa dovrebbe avere un **writer ordinario unico** oppure un meccanismo che serializzi le modifiche.

Serializzare significa, in sostanza, metterle in fila.

Immaginiamo due persone che devono correggere lo stesso contratto. Se entrambe lavorano contemporaneamente su copie differenti e poi sostituiscono il documento principale, una delle due può cancellare il lavoro dell'altra. Se invece le modifiche vengono applicate in sequenza sulla versione corrente, il conflitto diventa gestibile.

La regola non dice che tutto il WCM debba avere un unico scrittore. Dice che **la responsabilità di scrittura su uno stesso confine condiviso deve essere chiara**.

46.9 Quinto controllo: ripetere l'operazione non deve duplicarne l'effetto

Le operazioni remote possono essere ritentate. Una risposta può non arrivare, una connessione può interrompersi o un'automazione può rieseguire lo stesso passaggio.

Per questo la scrittura deve essere, quando possibile, **idempotente**.

Abbiamo già incontrato questo concetto: ripetere lo stesso fatto non deve creare un fatto nuovo.

Per esempio, registrare due volte lo stesso comando logico non dovrebbe produrre due autorizzazioni distinte soltanto perché la richiesta è stata ritentata.

STESSA INTENZIONE LOGICA
→ STESSA IDENTITÀ
→ NESSUN SECONDO EFFETTO ARTIFICIALE

Stable identity e idempotenza rendono retry e replay strumenti di resilienza invece che fonti di duplicazione.

46.10 Sesto controllo: verificare ciò che è stato davvero salvato

Un messaggio di successo della chiamata tecnica non è la prova finale dell'effetto.

PROT-017 richiede una **Post-Write Verification**: dopo la scrittura, il sistema deve verificare il valore realmente persistito quando il rischio lo richiede.

È una regola molto concreta.

Se inviamo una modifica importante e riceviamo “operazione riuscita”, non ci limitiamo al messaggio. Controlliamo che il documento, lo stato o la risorsa risultino effettivamente come previsto.

Solo allora l'effetto può essere accettato.

```
WRITE OK
→ VERIFY PERSISTED RESULT
→ MATCH?
  └─ YES → EFFECT ACCEPTED
  └─ NO  → EFFECT NOT ACCEPTED
```

Questa distinzione evita che un successo di trasporto venga confuso con un successo semantico e operativo.

46.11 Settimo controllo: una scrittura non può inventarsi più autorità

Una capacità tecnica di scrittura non conferisce authority.

Un componente può essere perfettamente capace di modificare dieci risorse, ma se l'autorizzazione riguarda una sola risorsa, le altre nove restano fuori scope.

PROT-017 impone quindi l'**Authority Boundary Preservation**:

la scrittura non può ampliare target, scope o potere decisionale rispetto alla fonte che la autorizza.

Questo principio è particolarmente importante per comandi e receipt di authority. Il fatto che un comando sia stato registrato non significa automaticamente che il suo effetto sia già stato eseguito. Il consumer deve verificare che il comando sia valido, corrente, compatibile con il target e non già consumato in modo equivalente.

La tecnica esegue. L'authority stabilisce **che cosa può essere eseguito**.

46.12 Un esempio quotidiano: modificare una prenotazione condivisa

Immaginiamo una prenotazione per una sala riunioni gestita da più persone.

Una persona apre la prenotazione delle 15:00 e decide di spostarla alle 16:00. Prima di salvare dovrebbe poter rispondere a una serie di domande:

1. è davvero quella la prenotazione da modificare?
2. il nuovo orario è espresso in una forma valida?
3. la prenotazione è ancora nello stato che avevo letto o qualcuno l'ha già cambiata?
4. un altro sistema la sta modificando contemporaneamente?

5. se premo due volte “salva”, creo una seconda prenotazione?
6. dopo il salvataggio, l'orario risulta davvero 16:00?
7. avevo il diritto di modificare quella prenotazione o soltanto di consultarla?

Queste sette domande corrispondono quasi esattamente ai guard principali di PROT-017.

Il protocollo non rende complicata una scrittura semplice. Rende espliciti i controlli necessari quando sbagliare avrebbe conseguenze persistenti.

46.13 Il flusso completo

La baseline del protocollo può essere riassunta così:

```
INTENZIONE / AUTHORITY
↓
TARGET E SCOPE ESATTI
↓
PAYLOAD / STRUTTURA VALIDI
↓
VERSIONE / STATO ATTESI ANCORA CORRETTI?
↓
WRITER OWNERSHIP / SERIALIZZAZIONE
↓
IDENTITÀ STABILE + WRITE IDEMPOTENTE
↓
SCRITTURA
↓
VERIFICA DEL RISULTATO PERSISTITO
↓
EFFETTO ACCETTATO
```

Ogni passaggio risponde a un tipo diverso di rischio.

Il target protegge dall'errore di destinazione. Lo schema protegge dall'errore di forma. L'expected state protegge dalla stale write. La serializzazione protegge dalla concorrenza. L'idempotenza protegge dai duplicati. La verifica protegge dai falsi successi. L'authority boundary protegge dall'estensione non autorizzata dell'effetto.

46.14 Protezione proporzionata al rischio

PROT-017 non trasforma ogni salvataggio in una cerimonia pesante.

La baseline distingue il rischio.

```
LOW RISK / LOCAL / EPHEMERAL
→ guard minimi coerenti

PERSISTENT / SHARED
→ target + struttura + expected state + verifica

CANON / AUTHORITY / DESTRUCTIVE
→ tutti i guard pertinenti + fail closed + provenance
```

La logica è importante perché un sistema troppo permissivo è fragile, ma un sistema che applica il massimo costo di controllo a qualunque gesto diventa inefficiente.

La sicurezza deve crescere con la materialità della conseguenza.

46.15 Sostituire un file remoto senza perdere contenuto

La sostituzione di un file esistente è un caso semplice da comprendere e molto utile per vedere il protocollo in azione.

Prima della sostituzione, la baseline richiede di:

- leggere il contenuto e la versione corrente;
- verificare il percorso e il confine corretto;
- costruire il contenuto completo che dovrà sostituire il precedente;
- evitare una sostituzione vuota o involontariamente distruttiva;
- usare, quando disponibile, la versione attesa come condizione della write;
- verificare nuovamente il risultato dopo il salvataggio.

La regola più importante è che una sostituzione non dovrebbe partire da una fotografia vecchia e poi essere forzata sulla realtà corrente.

Se nel frattempo la risorsa è cambiata, occorre rileggere e ricostruire l'operazione.

46.16 Writer automatici: l'automazione non elimina il rischio

Un'automazione può eseguire la stessa operazione centinaia di volte con grande velocità. Proprio per questo una regola sbagliata può propagarsi altrettanto velocemente.

Per i writer automatici su risorse condivise, PROT-017 richiede quindi di preferire un writer ordinario unico, serializzare le operazioni concorrenti e, in caso di collisione, rileggere la baseline corrente prima di ricalcolare l'operazione.

Il punto è sottile:

su collisione non si forza il vecchio payload; si ricostruisce l'operazione sullo stato corrente.

Inoltre, un errore tecnico di persistenza non deve trasformarsi automaticamente in una nuova decisione, una nuova authority o un cambiamento semantico del workflow.

Un problema di scrittura resta un problema di scrittura finché le fonti autorevoli non dicono altro.

46.17 Il caso speciale dei checkpoint di workflow

La baseline corrente contiene una protezione aggiuntiva per i checkpoint persistenti del workflow quando una modifica può produrre uno stato di attesa di authority o cambiare un Board Gate.

In questi casi non basta salvare un checkpoint e controllarlo dopo. Il payload completo deve superare il contratto previsto **prima** della scrittura.

Se il checkpoint non contiene tutti gli elementi necessari per rappresentare correttamente il gate, la write non dovrebbe diventare stato corrente soltanto perché il repository l'ha tecnicamente accettata.

Esiste inoltre una barriera di contenimento che può ricontrollare i checkpoint persistenti e, in caso di stato invalido, recuperare l'ultima versione storica valida oppure rimuovere dal current state un checkpoint per il quale non esiste una versione valida.

Ma il principio resta quello già visto:

la recovery è una seconda barriera; non sostituisce il controllo prima della scrittura.

46.18 Cosa succede quando un guard fallisce

PROT-017 preferisce fermare una modifica piuttosto che indovinare.

I principali failure mode sono molto leggibili:

TARGET AMBIGUO

→ NO WRITE

PAYLOAD INVALIDO

→ NO WRITE

VERSIONE ATTESA STALE

→ RELOAD / REPLAY

WRITER OWNERSHIP AMBIGUO

→ STOP / SERIALIZE

VERIFICA POST-WRITE FALLITA

→ NON ACCETTARE L'EFFETTO

AUTHORITY INSUFFICIENTE

→ NO WRITE FUORI SCOPE

Un eventuale rollback riuscito non cancella il fatto che sia avvenuta una failure. L'errore e il recovery restano evidence utile, con la propria provenienza.

Questo permette al sistema di imparare senza riscrivere la storia come se l'incidente non fosse mai avvenuto.

46.19 Cosa produce PROT-017

L'output desiderato del protocollo non è semplicemente “un file salvato”.

È una **mutazione persistente accettabile**, cioè una modifica per la quale il sistema possiede evidenza sufficiente che:

- il target fosse quello corretto;
- il payload fosse valido;
- la versione di partenza fosse ancora compatibile;
- la responsabilità di scrittura fosse coerente;
- retry e replay non producessero duplicazioni logiche;
- il risultato persistito corrispondesse a quello previsto;
- l'effetto non superasse l'autorità disponibile.

Se una di queste condizioni obbligatorie fallisce, il risultato tecnico della write non basta per dichiarare l'effetto accettato.

46.20 Relazione con gli altri protocolli

PROT-017 è una baseline trasversale di sicurezza. Non sostituisce i protocolli specializzati.

PROT-001 tratta la sicurezza del lavoro Git e del working tree locale.

PROT-004 tratta dispatch e idempotenza nel proprio ambito.

PROT-010 disciplina i comandi di authority della command surface e usa PROT-017 come safety baseline generale.

PROT-016, visto nel capitolo precedente, definisce il contratto per stato e projection deterministici.

PROT-017 entra nel punto in cui uno di questi meccanismi deve **persistere una modifica** e protegge quella transizione dal “prima” al “dopo”.

I protocolli specializzati possono richiedere controlli ancora più forti, ma non dovrebbero indebolire le protezioni pertinenti allo stesso rischio.

46.21 Maturity: cosa possiamo dire e cosa no

PROT-017 è una baseline **ACTIVE**, promossa a partire da esperienza metodologica registrata nel WCM. La sua origine è legata a evidenze concrete di scritture remote distruttive, regressioni sul target di authority e problemi di stale/concurrent writer, successivamente trasformate in una regola generale attraverso il processo di promotion previsto dal metodo.

Questo è importante perché il protocollo non nasce soltanto da una preferenza teorica.

Ma non significa che ogni possibile tecnologia, organizzazione o dominio sia stato validato universalmente.

La formulazione corretta è quindi:

- il protocollo è parte della baseline WCM corrente;
- le sue protezioni derivano da problemi osservati e da successiva formalizzazione;
- la loro applicazione concreta deve restare proporzionata al boundary e al rischio;

- non esiste una prova universale che ogni failure di persistenza possibile sia coperta dal protocollo.
-

46.22 La regola da ricordare

Se dovessimo conservare una sola idea di questo capitolo, sarebbe questa:

Una scrittura persistente non è sicura perché il sistema sa eseguirla. È sicura quando target, contenuto, versione, writer, identità, authority e risultato sono stati controllati in modo proporzionato al rischio.

Nel capitolo precedente abbiamo separato i fatti strutturati dall'interpretazione libera. Qui aggiungiamo il passo successivo: quando quei fatti devono essere modificati, la scrittura deve essere protetta prima di diventare realtà persistente.

Source Map essenziale

Fonte primaria:

- [wcm/process-book/protocols/PROT-017_PERSISTENT_MUTATION_SAFETY.md](#) — ACTIVE / PROMOTED FROM WCM-LRN-004.

Fonti collegate richiamate dalla baseline del protocollo:

- [wcm/process-book/processes/PROC-004_EVIDENCE_TO_BASELINE_PROMOTION.md](#);
- [wcm/process-book/protocols/PROT-010_MISSION_CONTROL_AUTHORITY_COMMAND.md](#);
- [wcm/process-book/protocols/PROT-016_DETERMINISTIC_STATE_PROJECTION.md](#);
- [wcm/kb/learning/records/WCM-LRN-004_REMOTE_WRITES_NEED_PAYLOAD_GUARDS.md](#).

Maturity qualifier: baseline WCM attiva, promossa da evidence metodologica; nessun claim di validazione universale.